Daily Epsilon of Maths 11 Nov 2026

Problem

Find the first two digits after the radix point in π in base 8.

Solution

We will write a haskell type for approximating real numbers using rational numbers, providing access to the current lower bound and upper bound as rational numbers, and a method to make the bounds tighter. Then, we will define π by a value of that type, using some known approximation algorithm. Then, we will repeatedly try to determine π to 2 octal places, making the approximation more precise if we fail to determine the digits precisely.

In particular, I will use the Gregory-Leibniz series approximation

$$\pi = \sum_{i=0}^{\infty} (-1)^i \frac{4}{2i+1}$$

as it is easy to code. Defining the partial sums

$$s_n = \sum_{i=0}^{n} (-1)^i \frac{4}{2i+1}$$

with $s_{-1} = 0$, we know that for all odd n we have $s_n < \pi$ and for all even n we have $\pi < s_n$, and we can easily determine the next partial sum by the formula $s_n = s_{n-1} + (-1)^n \frac{4}{2n+1}$. Thus, we will start with lower bound $s_{-1} = 0$ and upper bound $s_0 = 4$, and whenever we need to tighten the bounds, we compute the next partial sum and update the corresponding bound to that.

Code

```haskell
import Control.Monad.ST (ST, runST)
import Data.STRef (STRef, newSTRef, readSTRef, modifySTRef)
import Data.Function (fix)
import Numeric (showIntAtBase)

-- Storage for an approximation
data ApproxStore = ApproxStore
    { lowerBoundS :: Rational -- Current lower bound
    , upperBoundS :: Rational -- Current upper bound
    , tightenS :: ApproxStore -- Next approximation
    }

type Approximation s = STRef s ApproxStore

-- Get current lower bound
lowerBound :: Approximation s → ST s Rational
lowerBound x = lowerBoundS <$> readSTRef x

-- Get current upper bound
upperBound :: Approximation s → ST s Rational
upperBound x = upperBoundS <$> readSTRef x

-- Tighten bounds
tighten :: Approximation s → ST s ()
tighten x = modifySTRef x tightenS

{-
    Round an approximation down to some multiple of a rational number, making
    the approximation more precise if needed to determine the result
-}
roundDown :: Rational → Approximation s → ST s Rational
roundDown epsilon x = do
    let
        multiples :: Rational → Integer
        multiples r = floor $ r / epsilon
    lbmul ← multiples <$> lowerBound x
    ubmul ← multiples <$> upperBound x
    if lbmul == ubmul then
        pure $ fromInteger lbmul * epsilon
    else do
        tighten x
        roundDown epsilon x

-- Gregory-Leibniz approximation of pi
approxPi :: ST s (Approximation s)
approxPi = newSTRef $ go 0 4 1 where
    go :: Rational → Rational → Integer → ApproxStore
    go l u n = ApproxStore l u $
        if even n then
            go l (l + 4 / (2 * fromInteger n + 1)) (n + 1)
        else
            go (u - 4 / (2 * fromInteger n + 1)) u (n + 1)

{-
    Show an approximation to a given amount of places to the right of the radix
    point of an approximation in a given base
-}
-- Base should be between 2 and 9 or weird stuff will happen
digits :: Integer → Int → Approximation s → ST s String
digits base numplaces x = do
    let
        epsilon = (1 / fromInteger base) ^ numplaces
    rounded ← roundDown epsilon x
    let
        wholePart = floor rounded
        fracPart = rounded - fromInteger wholePart
        wholeDigits = show wholePart
        fracDigitsUnpadded = showIntAtBase
            base
            (head . show)
            (floor $ fracPart / epsilon)
            ""
        fracDigits =
            replicate (numplaces - length fracDigitsUnpadded) '0' <>
            fracDigitsUnpadded
    pure $ wholeDigits <> "." <> fracDigits

main :: IO ()
main = putStrLn $ runST $ approxPi >>= digits 8 2
```

Result

```
3.11
```